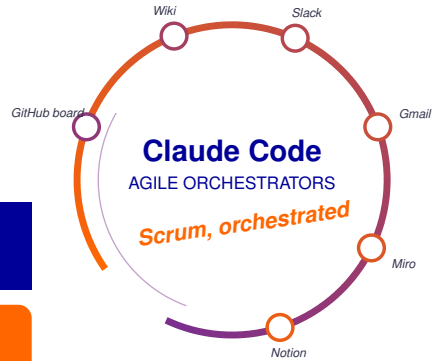
Agile Orchestrators

How a modern AI harness changes scrum work in a dev team

Team Agile Orchestrators · Tulip Churn case study



Rabobank

Agenda

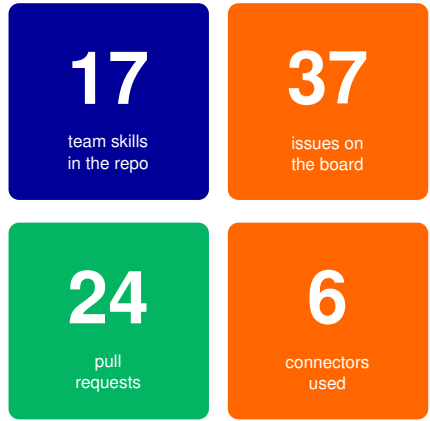
What we will cover

00	*The experiment*	Tulip Bank, run like a real team with an AI harness
01	*Skills as a quality gate*	Rules and templates applied every time
02	*Less friction*	Scrum operations and maintenance by prompt
03	*Documentation that stays current*	Rules and docs fixed as part of the work
04	*Is it faster?*	Scrum: yes. Coding: context decides
05	*SlopGuard*	A side quest against AI slop
06	*What it means for Rabobank*	Confluence, Azure DevOps, Copilot and a proposal

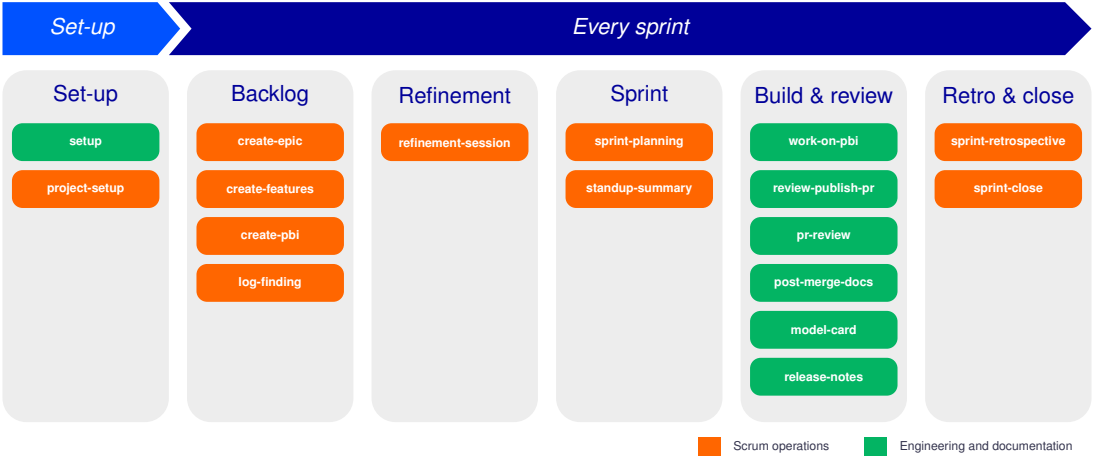
Tulip Bank: a dummy project, run like a real team

Agile Orchestrators set out to show how a modern AI harness changes the daily scrum work of a dev team.

- **The case:** *Tulip Bank churn early-warning*: synthetic data, a scikit-learn model, a FastAPI service.
- **A real scrum setup:** GitHub Project board, sprints, story points, a wiki with Definition of Ready, Definition of Done and ADRs.
- **A real disruption:** mid-sprint, the previous team left and four new full-time engineers joined.
- **One rule:** every scrum operation went through Claude Code, and each one was logged, misses included.



One track through the whole scrum lifecycle

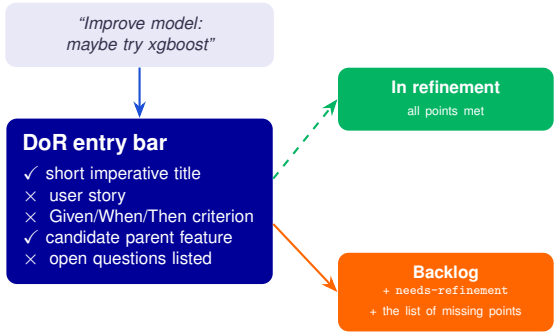


Discipline moves from people into the skill

Every team has a Definition of Ready. Applying it to *every* item, *every* sprint, under time pressure, is the hard part.

A **skill** is a written workflow (rules, templates and checks) that the harness runs the same way every time.

- **Templates:** every PBI gets a story, Given/When/Then criteria and an estimate.
- **Gates:** an item that misses the entry bar stays out of *In refinement*, with the missing points listed.
- **Checks:** before a PR, the branch is reviewed against the acceptance criteria and the DoD.



PBI #11 as it was on the board

Quality went up on every step of scrum

Backlog items

Every PBI starts from the template: story, Given/When/Then, technical notes, estimate. Findings go to Backlog with `needs-refinement`, never straight to refinement.

Refinement

#11 “maybe try xgboost” became a story with a named metric (precision in the top 10 %, the ~500 customers Retention calls each week), 4 criteria and 5 SP. Duplicate #12 closed.

Sprint planning

Last sprint’s leftovers against Ready items, sized to the capacity of the new team. Deadlines and risks are now surfaced explicitly.

Build and review

The branch is checked against the acceptance criteria and the DoD before the PR. The reviewer is picked from who last touched the files.

Sprint close

A read-only dry run found 3 Done items with a DoD gap: two merged with changes requested, one merged without an approval.

Retrospective

Talking points backed by facts: 19 of 28 commits on `main` since 28 Sep had no PR. The retro board was built on Miro from these facts.

Scrum operations by prompt

“Move it to Ready.” “Put it In progress, not closed.”	Looked up the Project field and option ids and set Status, Sprint, Priority and Story Points through the Projects API.
“Give the new team the state of the board.”	Read every open issue on the board and summed it up for the four new engineers.
“Colleagues cannot see the board.”	Found that repo access is not project access, and added every repo contributor to the project.
“Create the epic and the feature, and link the PBIs.”	Created Epic #27 and Feature #28 and linked the PBIs as sub-issues with the matching labels.
“Make squash the only merge method.”	Disabled merge commits and rebase merges on the repo and added the rule to `CLAUDE.md`.

Friction removed, control kept

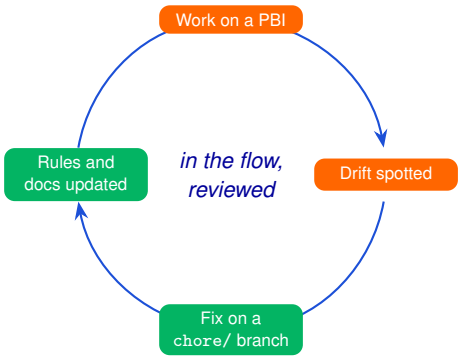
The harness does

- Board fields, iterations, sub-issues and labels, on request.
- `/setup` takes a newcomer with only Claude Code and a clone to a working environment: `gh`, `uv`, the wiki clone and the connectors.
- Finds the Slack invite in the PO's Gmail, creates the private channel and sends a test message.
- Builds the retrospective board on Miro from the sprint's facts.
- Picks the reviewer from the git history and requests the review.

The human decides

- Creating accounts: the PO created the Slack account, the one step Claude may not do.
- Logging in to connectors (OAuth), e.g. `/mcp` for Miro.
- Destructive actions: deleting 8 merged branches waited for the PO's approval.
- Sharing boards, and the per-ticket decisions at sprint close.
- Plans before work: the PO approved the harness's plan for #11.

Rules and docs are fixed as part of the work



- **Rules as code.** `CLAUDE.md` and the skills are versioned and reviewed in PRs like any other change. The Definition of Ready on the wiki and the entry bar in the skills were aligned.
- **Migration without leftovers.** The docs moved from Notion to the GitHub Wiki (#24). Then `/setup` and the README were cleaned of Notion, so newcomers are no longer sent to a tool nobody uses.
- **Wrong instruction, fixed at once.** The documented wiki push failed (HTTPS clone, SSH login). PR #60 made `/setup` configure it.
- **Decisions from measurements.** ADR-002 “keep gradient boosting” was written from the model comparison and moved from Proposed to Accepted.
- **After every merge,** `post-merge-docs` checks which docs the change made stale.
- **This talk:** every operation was logged by the harness as it happened.

Faster? An honest answer

Scrum operations	**Clearly yes**

- Board changes, refinement, planning preparation and the retro board take a prompt instead of clicks, forms and copy-paste.
- A newcomer with only Claude Code gets a working setup through `/setup`.
- Reviews, findings and docs happen inside the flow instead of as extra chores.

Coding	**Hard to evaluate**

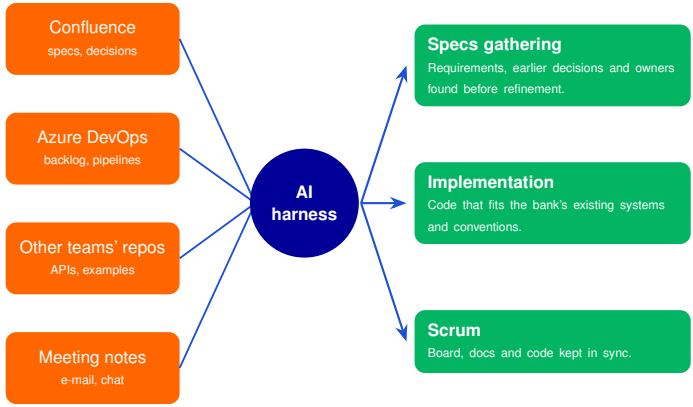
- A dummy project with synthetic data: the tasks were not hard.
- #11 “try XGBoost” was answered with a measurement. Current model 0.801 AUC; the formula that generated the data scores 0.810. XGBoost 0.799. No model change.
- The gain came from asking *how much room is left* before building, more than from typing speed.

The real multiplier is context

On Tulip Churn, the context was small: one repo, one board, one wiki.

In the bank, the context is spread over many Confluence spaces, Azure DevOps projects, repos and teams, and finding the right piece takes time.

A harness that can **navigate** that context could speed up much more than scrum operations.



Assumption, not measured in this project: to be tested in a pilot.

AI writes code fast, and slop piles up just as fast

AI slop: code that works but is too long, too branchy, duplicated, dead or over-typed. Each piece looks fine in review. Together they make the codebase harder to change.

With an agent that writes and commits on its own, reviewers cannot catch all of it.

SlopGuard is a Claude Code hook we built as a side quest: it stops Claude from finishing while the code contains slop, and tells it what to fix.

Open source:
github.com/kimzed/python-slopguard

Too complex	complexity, arguments, statements, nesting	`ruff`
Too long	files over a line limit	`built in`
Type slop	deeply nested annotations, wide tuples	`built in`
Dead code	anywhere in the configured paths	`vulture`
Copy-paste	duplicated blocks that involve a changed file	`PMD CPD`

What dogfooding SlopGuard taught us

1 *A green check with nothing checked*

The Stop hook only looked at uncommitted files. Claude committed its work during the turn, so at Stop the tree was clean and everything passed unchecked.

Fix: a `PreToolUse` hook now runs SlopGuard before every `git commit` Claude makes.

2 *The rerun*

#11 was rebuilt from scratch with the commit hook live. Two commits, both genuinely checked, no blocks.

`compare.py` came out as 13 small functions, with the settings in a dataclass.

3 *What a linter cannot see*

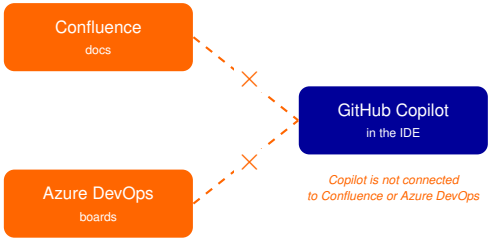
Unseeded models: XGBoost scored 0.799 AUC on one run and 0.762 on the next. Found by re-running, not by the guard.

PMD only matches exact copies, so renamed copy-paste slips through.

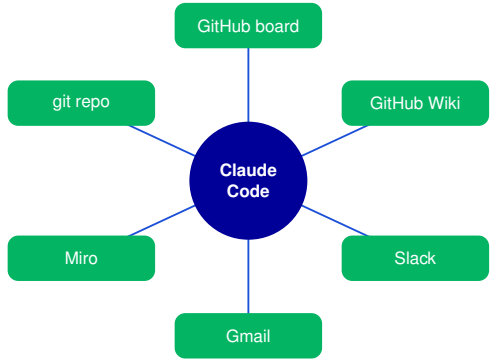
Two issues were opened upstream on SlopGuard from this project.

Our tools at the bank do not talk to each other yet

Rabobank today



In this project



GitHub board ≈ Azure DevOps Boards · GitHub Wiki ≈ Confluence

The connections made the difference.

Proposal: bring the harness to a Rabobank team

Pilot track

1 *Connect*

Give the team's AI assistant access to Azure DevOps and Confluence, through connectors, with the bank's security controls.

2 *Encode*

Turn our Definition of Ready, Definition of Done, PBI template and ceremonies into shared skills that live in the repo.

3 *Guard*

Put SlopGuard, or an equivalent gate, on AI-written code from day one, with limits agreed by the team.

4 *Pilot*

One real team. Compare refinement quality, cycle time and documentation freshness with today's baseline.

Takeaways

1	*Skills are a quality gate*	Rules and templates applied to every item, every sprint, with little discipline needed from the team.
2	*Less friction*	Board, repo and tool admin by prompt. People keep the decisions.
3	*Docs that stay current*	Rules live as code and get fixed as part of the work.
4	*Faster*	Clearly for scrum. For coding, bank-wide context is the multiplier: to be tested.
5	*SlopGuard*	Keeps AI-written code small, simple and free of dead or copied code.

Questions?

Thank you from
Team Agile Orchestrators

Repo, board, wiki and the harness log are open to browse



Rabobank